

Enhancing Model Checker Test Generation with Functional Programming

Charles O'Malley, BAI

A Thesis

Presented to the University of Dublin, Trinity College

in partial fulfilment of the requirements for the degree of

Master of Science in Computer Science (Computer Engineering)

Supervisor: Andrew Butterfield

April 2026

Enhancing Model Checker Test Generation with Functional Programming

Charles O'Malley, Master of Science in Computer Science
University of Dublin, Trinity College, 2026

Supervisor: Andrew Butterfield

This dissertation explores whether the refinement stage of an existing SPIN/Promela-based RTEMS test-generation pipeline can be moved from Python to Haskell without changing the behaviour of the generated tests. The work focuses on `refine` command, a stateful part of the system that turns annotated SPIN traces into C test code. Instead of replacing the whole pipeline at once, the project uses a bridge-based approach so that the Haskell version can be introduced gradually and checked against the original Python implementation. The port eventually moved away from line-by-line translation and towards a more Haskell-native design based on explicit state and smaller transformations. The two versions were then compared across the seven active models in the repository. This process found one real bug in the Haskell output, traced to template brace handling, which was fixed. After that, the remaining differences were limited to harmless formatting. Overall, the project shows that the refiner can be ported in a behaviour-preserving way.

Acknowledgments

Thank you Mum & Dad.

CHARLES O'MALLEY

University of Dublin, Trinity College
April 2026

Contents

Abstract	i
Acknowledgments	ii
Chapter 1 Introduction	1
1.1 Motivation	1
1.2 Aim and Scope	1
1.3 Dissertation Structure	2
Chapter 2 State of the Art	3
2.1 Model Checking and Test Generation	3
2.2 Functional Programming and Haskell	4
2.3 Multi-Language Systems	4
2.4 Closely Related Work	5
Chapter 3 Design	7
3.1 Problem Formulation	7
3.1.1 Identified Challenges	8
3.1.2 Proposed Work	8
3.2 Overview of the Design	8
3.3 Summary	9
Chapter 4 Implementation	10
4.1 Overview of the Solution	10
4.2 The Implementation	11
4.2.1 The Bridge	11
4.2.2 The Diff Script	11
4.2.3 Refine Command	12
4.3 Summary	14

Chapter 5	Evaluation	15
5.1	Experiments	15
5.2	Results	15
5.3	Summary	17
Chapter 6	Conclusions & Future Work	18
6.1	Conclusions	18
6.2	Future Work	19
Bibliography		20

List of Tables

2.1	Context for the dissertation	6
5.1	Comparison results across the seven models	16

List of Figures

Chapter 1

Introduction

This chapter introduces the dissertation, motivates the decision to port part of the RTEMS test-generation pipeline from Python to Haskell, and explains the structure of the document. It first describes the technical context of the project, then states the aim and scope of the work, and finally shows how the remaining chapters support the overall argument.

1.1 Motivation

The repository used in this project contains a SPIN/Promela-based workflow for generating RTEMS validation tests. Promela models are explored with SPIN, the resulting annotated traces are refined into concrete test actions, and those actions are emitted as C test files. The stage studied here is the refinement step, implemented historically in Python. That step is small enough to replace in isolation, but central enough that any change to it can affect the entire pipeline.

The motivation for the project came from the structure of the existing refiner rather than from a failure of the generated tests. The Python version already generated useful RTEMS tests, but it accumulated mutable state, branching logic, and template expansion in one place. That made it harder to reason about the code, and it made larger structural changes risky. Haskell was therefore investigated not as a way to generate different tests, but as a way to re-express the same refinement logic with clearer state handling and stronger separation between stages. 

1.2 Aim and Scope

The main aim of the dissertation is to determine whether the `refine_command` stage can be ported from Python to Haskell while preserving the behaviour of the existing generator.

In practical terms, that means the Haskell version should accept the same annotated SPIN output and produce equivalent test code for the existing RTEMS models.

The scope of the work is deliberately narrow. The dissertation does not attempt to redesign the full `testbuilder` workflow, replace RTEMS itself, or prove that Haskell is universally better than Python for test generation. Instead, it focuses on one difficult, stateful part of the toolchain and studies how a gradual port can be carried out without losing confidence in the output.

The main contributions of the dissertation are:

- a bridge-based strategy for introducing a Haskell refiner without discarding the existing Python pipeline;
- an explicit-state Haskell implementation of the refinement stage;
- an empirical comparison of Python and Haskell generated outputs across the seven active Promela models in the repository.

1.3 Dissertation Structure

Chapter 2 reviews the background needed for the project, including model-checker-based test generation, functional programming, and multilingual system design. Chapter 3 defines the problem more precisely and explains the design decisions behind the bridge-based port. Chapter 4 describes how the refiner was moved from Python to Haskell and where the implementation changed from line-by-line translation to a more Haskell-native design. Chapter 5 evaluates the result by comparing generated outputs and discussing the remaining limitations of the workflow. Chapter 6 closes the dissertation and identifies the most useful directions for further work.

Chapter 2

State of the Art

This chapter positions the dissertation within the background work that made the project possible. It first reviews model-checker-based test generation and the RTEMS-specific context, then discusses the functional programming ideas that motivate the use of Haskell, and finally considers the practical problems of combining Python and Haskell inside one toolchain.

2.1 Model Checking and Test Generation

Model checking is normally presented as a verification technique, but in this project it is used in a more operational way. SPIN explores the Promela model, produces counterexample traces when assertions are violated, and those traces are then treated as structured test scenarios rather than as the final result. The important point for this dissertation is that the generated test cases do not come directly from the model checker alone. They depend on a second translation stage that maps annotated SPIN output onto RTEMS-oriented test code.

That distinction matters because the dissertation does not replace the modelling or exploration stages. The Promela models, SPIN search, and RTEMS test structure already existed. The difficult engineering problem is the refinement step in the middle: reading annotation lines such as `CALL`, `STATE`, `STRUCT`, and `SEQ`, tracking enough context to interpret them correctly, and emitting the right C fragments in the right order.

A broader example of the same general idea appears in the work of Zheng et al. (Zheng et al., 2007). They use model checking to derive tests for service behaviour described in BPEL-style models. The domain is different from RTEMS, and the modelling formalism is different from the Promela workflow used here, but the wider pattern is similar: model checking is used to systematically derive execution scenarios, and a second stage turns those scenarios into executable tests. That makes this dissertation easier to place in a

broader test-generation tradition rather than treating it as an isolated RTEMS-specific toolchain.



2.2 Functional Programming and Haskell

A useful starting point for the Haskell side of the project is Hughes’s argument that modularity is one of the strongest practical benefits of functional programming (Hughes, 1989). That is directly relevant here because the refiner is a transformation stage: it reads one representation of behaviour and incrementally builds another. A style that makes intermediate state and data flow explicit is therefore attractive, especially when behaviour must be preserved while the internal structure changes.

That argument is useful here mainly because the refiner is full of small, state-dependent transformations. The code does not need a language chosen for novelty; it needs a language whose normal style encourages clearer composition and more local reasoning. In that sense the appeal of functional programming in this dissertation is practical rather than abstract. The main benefit is not that the program becomes mathematically elegant, but that the steps by which one annotation line changes the next runtime state can be expressed more directly.

The project also draws on the general history and design goals of Haskell, summarised by Hudak et al. (Hudak et al., 2007). Haskell’s combination of strong static typing, algebraic data types, pattern matching, and purity by default makes it well suited to programs that perform structured translation. Those properties do not automatically make the code correct, but they encourage an implementation style in which cases are more explicit and state changes are easier to trace.

That history is also relevant because it frames Haskell as a language designed to support reusable abstraction and a clear common standard, rather than as a niche experiment. For this dissertation that matters because the port is not trying to prove that Haskell is universally better than Python. It is simply using a language whose usual tools fit a translation problem well: explicit data modelling, case analysis, and a clearer separation between pure transformation and boundary-facing input and output.



2.3 Multi-Language Systems

The project is also part of a broader class of multilingual systems, where one language is introduced into an existing codebase rather than replacing it completely. A recent empirical study by Yang et al. (Yang et al., 2024) highlights interface boundaries, data conversion, and error diagnosis as recurring sources of friction. Those concerns match the

experience of this project closely  the main difficulty was not writing isolated Haskell functions, but deciding how Python and Haskell should communicate without making the combined system more fragile than the original one.

This problem shaped the eventual architecture. Instead of a deep foreign-function interface, the project used a narrow subprocess protocol over standard input and output. That choice keeps the boundary explicit. Configuration is passed in as JSON, annotation lines are refined one at a time, and the rendered test body is returned at the end. The approach is less elegant than a direct import, but it is easier to inspect, easier to debug, and better aligned with the existing Python pipeline. It also reflects the main lesson of the multilingual-systems literature in a fairly direct way: once the boundary is explicit, it becomes easier to reason about what data crosses it and where failures should be diagnosed.

Two concrete integration options were considered during the project. The HaPy library (Fisher, 2014) suggested a more direct Python-to-Haskell interface by exposing compiled Haskell code to Python, but its documented setup depends on shared-library builds and a limited set of supported exported types. The call-haskell-from-anything library (Hambüchen, 2011) explored a more general MessagePack-based boundary over the C foreign-function interface. Both were useful as engineering reference points, but neither was adopted. The repository needed a controlled way to preserve the existing Python API while changing the implementation behind it, and the simpler subprocess boundary turned out to be the safer fit for that requirement.



2.4 Closely Related Work

The most directly relevant prior work is the earlier dissertation by Jaškuć (Jaškuć, 2022). That work is important because it establishes the value of the underlying RTEMS/SPIN framework itself. The present dissertation does not need to justify the entire modelling approach from first principles; it builds on an existing system that had already shown that Promela models and SPIN traces could be refined into useful RTEMS tests.

The difference in emphasis is therefore clear. The earlier work extended the range and usefulness of the RTEMS modelling framework. This dissertation instead studies the maintainability of one critical part of that framework by replacing the refinement stage with a Haskell implementation. The contribution is not a new manager model or a new coverage result, but a controlled port of the translation logic used by the existing models.

Source	Main Idea	Relevance Here
<i>Why Functional Programming Matters</i> (Hughes, 1989)	Functional programming improves modularity and composition	Motivates the move from implicit mutation toward explicit transformations
<i>A History of Haskell</i> (Hudak et al., 2007)	Explains the design goals of Haskell as a common non-strict functional language	Justifies Haskell as a plausible replacement language for a structured translation stage
Yang et al. (Yang et al., 2024)	Identifies interface boundaries and data conversion as common developer difficulties	Motivates the explicit subprocess bridge between Python and Haskell
Zheng et al. (Zheng et al., 2007)	Uses model checking to derive executable tests from behavioural models	Shows that turning model-checker output into tests is a broader pattern, not just an RTEMS-specific one
Jaškuć (Jaškuć, 2022)	Demonstrates the value of the RTEMS/SPIN test-generation framework	Provides the immediate technical context that this dissertation extends

Table 2.1: Background sources and closely related work that motivate the technical choices made in the dissertation.

In summary, the literature and engineering context suggest two main conclusions. First, the project is well motivated as a study of how model-checker output can be preserved while the refinement stage is restructured. Second, the challenge is not only one of programming language choice, but also one of managing a clear interface between old and new implementations.



Chapter 3

Design

This chapter turns the background of the previous chapter into concrete design decisions. It defines the specific problem within the existing RTEMS toolchain, identifies the constraints that any replacement must satisfy, and explains the bridge-based design used to introduce Haskell gradually rather than by one large swap.

3.1 Problem Formulation

The problem addressed by this dissertation is not that the existing generator failed to produce useful RTEMS tests. The problem is that one central stage of the pipeline, the Python refiner, became difficult to restructure safely as the project grew. The `testbuilder` workflow already had working Promela models, a working SPIN-based exploration stage, and working C test generation. Changing the refiner therefore meant changing the part of the system where the risk of regression was highest.

The existing Python refiner combines several kinds of work in one place. It parses annotation lines, tracks context such as whether refinement is currently inside a `STRUCT` or `SEQ`, performs lookups in a refinement dictionary, and accumulates generated code fragments. This arrangement works, but it makes it harder to reason locally about how one line of input affects the next state of the translation.

Three properties of the original implementation were especially important. First, context is stored as mutable object state. Second, the main refinement function handles many annotation forms in one place. Third, template expansion and code accumulation are interleaved with control flow. Each of these is manageable in isolation, but together they make structural changes riskier than they need to be.

3.1.1 Identified Challenges

The first challenge was behavioural preservation. The repository already contained working models and a working Python refiner. A replacement could only be justified if it produced the same observable output for the existing model set.

The second challenge was integration. The refiner is not a standalone tool. It is called from the existing Python pipeline and depends on YAML refinement dictionaries, language templates, and per-model header fragments. Any design therefore had to fit around those existing inputs rather than invent a new workflow.

The third challenge was scope control. Replacing the entire pipeline at once would have made it difficult to distinguish language-porting issues from unrelated tooling issues. A narrower design was needed so that the hardest stateful component could be rewritten first and checked continuously against the original behaviour.

3.1.2 Proposed Work

The design adopted in this dissertation is a staged port of `refine_command`. The Haskell implementation keeps the same external role as the Python one: it receives tokenised annotation lines, updates its refinement state, and eventually renders a test body. The surrounding Python code is retained so that existing configuration files, model assets, and generation scripts can continue to drive the process.

The central design decision is therefore not only to use Haskell, but to introduce it behind a bridge that preserves the historical Python-facing interface. That keeps the migration incremental. The new implementation can be checked with the same inputs as the old one, and output differences can be detected before they spread into the rest of the workflow.

3.2 Overview of the Design

The overall design separates the system into three logical parts: SPIN produces annotated traces from Promela models, the bridge passes those annotations and configuration data to the Haskell refiner, and the existing Python-side tooling assembles the final test files around the rendered body. This decomposition isolates the port to the refinement stage while leaving the rest of the pipeline stable.

A line-oriented protocol was used between Python and Haskell because it made the interface explicit and easy to inspect. The bridge sends an initial JSON configuration, then a sequence of refinement commands, and finally requests the rendered test body. The protocol is simple enough to log and replay, which proved useful during debugging.

```
INIT JSON:{...}  
COMMAND:setupLanguage  
COMMAND:refineSPINLine:["4", "CALL", "WaitForSuspend", "2", "resumeRC"]  
COMMAND:testBody
```

Listing 3.1: Simplified bridge protocol between Python and Haskell

This design also made differential checking a first-class part of the project rather than an afterthought. Because the Haskell refiner could be exercised with the same traces as the Python original, text diffs on generated tests became the main feedback loop. That in turn made it possible to move from a line-by-line translation strategy to a more Haskell-native design without losing confidence in the observable output.

3.3 Summary

In summary, the design of the dissertation is conservative at the system boundary and experimental inside the refiner itself. The boundary stays stable so that behaviour can be checked continuously, while the implementation behind that boundary is free to adopt a clearer Haskell structure.



Chapter 4

Implementation

This chapter describes how the design of the previous chapter was realised in the repository. It explains the staged transition from the original Python refiner to the Haskell version, the supporting bridge and diff-based workflow used to keep the port testable, and the point at which the implementation stopped being a direct translation and became a more Haskell-native rewrite.

4.1 Overview of the Solution

The implementation followed a gradual transition from the original Python refiner to a Haskell refiner while keeping the rest of the pipeline recognisable. The main files involved were `refine_command.py`, `spin2test.py`, `combined_haskell_bridge.py`, and the new `refine_command.hs`. The objective was not to replace everything at once, but to replace the refinement stage in a way that could be checked end to end against the old behaviour.

A full rewrite from the start would have been risky. The refiner is small enough to look manageable, but it contains many state-dependent branches and many points where template expansion affects the final test file. Reaching the end of a full rewrite before comparing outputs would therefore have made it difficult to localise errors. The staged approach reduced that risk by keeping differential comparison in the loop throughout the port.

The testing and debugging effort was directed toward functional equivalence with the Python generator. The Python output already matched the expected RTEMS test structure, so the main implementation question was whether the Haskell version could preserve that observable behaviour while using a clearer internal structure.



4.2 The Implementation

4.2.1 The Bridge

During the transition, a bridge layer preserved the historical Python-facing `command` interface while moving the refinement work behind a subprocess boundary. Python remained responsible for orchestrating the wider pipeline, but Haskell could now be exercised inside that pipeline without forcing an immediate rewrite of every caller.

Communication was carried entirely through standard input and output. The bridge sends configuration data to the Haskell process, forwards annotation lines one at a time, and finally requests the rendered test body. This arrangement is deliberately plain, but it has two practical benefits: the interface is explicit, and the same messages can be logged or replayed when something goes wrong.



4.2.2 The Diff Script

A constant part of the porting process was checking the generated test files against the ones produced by the original system. At first this was done manually with an online text comparison tool, but that approach became unworkable once a single change could affect many generated files. A small local diff script therefore became part of the working process.

```
--- diff for tr-task-mgr-model-16.c ---
--- /home/a/Documents/Python_gen/tr-task-mgr-model-16.c
+++ /home/a/Downloads/rtems/RTEMS-SMP-Formal-main/formal/promela/models/task-
    mgr/gen/tr-task-mgr-model-16.c
@@ -177,10 +177,10 @@

    T_log(T_NORMAL,"@@@ 4 CALL WaitForSuspend 2 resumeRC");
    T_log( T_NORMAL, "Waiting for Task(%d) to Suspend", 2, resumeRC );
- do {
+ do {{
    resumeRC = ( *ctx->t_isSuspend )( 2 ? taskID[2] : 0xffffffff );
    rtems_task_wake_after( 100 );
- } while (resumeRC != RTEMS_ALREADY_SUSPENDED);
+ }} while (resumeRC != RTEMS_ALREADY_SUSPENDED);

    /* SWITCH[2] Suspension of proc4 in favour of proc5 */
    /* SWITCH[3] ReleaseTestSyncSema of proc4 (sometime) after proc5 */
@@ -370,6 +370,7 @@
```

}

Listing 4.1: Example output from the diff workflow showing a generated file diverging from the reference output. In practice, even a single extra line was worth investigating because small formatting mismatches could indicate a deeper error in template expansion.



The diff-based approach was intentionally strict. It reported trivial whitespace differences as well as genuinely important mismatches. That occasionally produced noise, but it was still useful because it made it harder for a small divergence to go unnoticed and later turn into a larger behavioural discrepancy. In practice, most debugging started from a diff, then moved backwards through the relevant refinement dictionary entry and the code path that emitted it.

Unit tests were also added around the Haskell refiner, but they were less decisive than the end-to-end diff checks. Unit tests were useful for protocol handling, configuration loading, and small local behaviours. They were less effective for the interactions that actually mattered most, where several pieces of state and several refinement templates combined to produce one incorrect line of generated code.

4.2.3 Refine Command

This was the main file to be converted. Its job is to take annotated SPIN output and refine it into concrete test code by matching annotation lines against entries in a model-specific refinement dictionary. Those entries are small templates of C code with placeholders for names, values, and identifiers taken from the trace.

At the beginning of the port, the Python code was translated into Haskell as closely as possible. The intention was to keep behaviour stable by keeping the structure stable. What became clear quite quickly, however, was that this strategy made the Haskell version look like Python written in another syntax. It preserved the awkward parts of the original without gaining much clarity from the new language.

The turning point came in state handling. The program has to remember whether the current annotation is inside a structure, inside a sequence, or part of ordinary top-level output. The Python version manages this with mutable object fields that are updated as each line is processed. That behaviour can be copied in Haskell, but copying it directly would have reproduced the same difficulty of following how state changes over time.

The more Haskell-native approach was to make the state explicit. Instead of mutating fields on one long-lived object, each stage receives a runtime state, transforms it, and returns the next one. At that point the port stopped being a line-by-line translation and became a Haskell implementation that happened to preserve the same outputs. The

aim from then on was not structural similarity to the Python source, but behavioural equivalence of the generated tests.

This change is visible in the following snippets.

Python state stored and mutated on the object

```
self.inSTRUCT = False
self.inSEQ = False
self.seqForSEQ = ""
self.inName = ""

case [pid, "SEQ", name]:
    self.inSEQ = True
    self.seqForSEQ = ""
    self.inName = name

case [pid, "SCALAR", "_", value]:
    self.seqForSEQ = self.seqForSEQ + " " + value
```

Listing 4.2: Python state stored and mutated on the object

In the Python version the state lives on the command object. As each SPIN line is read these values are updated in place, and later branches depend on whatever earlier branches left behind.

Haskell state made explicit

```
data RuntimeState = RuntimeState
{ rtInSTRUCT :: Bool
, rtInSEQ :: Bool
, rtSeqForSEQ :: Text
, rtInName :: Text
, rtDefCode :: [Text]
, rtDeclCode :: [Text]
, rtTestCodes :: [(Int, [Text])]
, rtCurrentId :: Int
, rtSwitchNo :: Int
}
```

Listing 4.3: Explicit runtime state in the Haskell refiner

The same information is still present, but it is now grouped into one explicit record. Rather than hidden mutation on an object, each function takes a state and returns the next one.

One SPIN line producing the next state

```

refineSPINLineMain :: [Text] -> RuntimeState -> RuntimeState
refineSPINLineMain ln rt0 =
    let rt1 = collectPIDsTokens ln rt0
        rt2 = if outputSWITCH (rtFlags rt1) then switchIfRequired ln rt1 else rt1
        rt3 = if annotateComments (rtFlags rt2) then logSPINLine ln rt2 else rt2
    in refineSPINLine ln rt3

```

Listing 4.4: Processing one SPIN line by returning a new state

This is the clearest expression of the change in approach. Instead of one long-lived object being updated in place, each stage produces a fresh version of the runtime state before the next stage runs.

Sequence handling as an explicit transition

```

_pidTxt:"SEQ":name:[] ->
    rt { rtInSEQ = True, rtSeqForSEQ = "", rtInName = name }

_pidTxt:"SCALAR":_:value:[] ->
    rt { rtSeqForSEQ = rtSeqForSEQ rt <> " " <> value }

```

Listing 4.5: Sequence state handled as an explicit transition

Sequence handling was where this change felt most useful. In this context a sequence is an ordered run of values collected between **SEQ** and **END** and later emitted through a matching `_SEQ` template. In the Haskell version, entering that mode and accumulating those values are both visible directly in the returned state.

4.3 Summary

The implementation was successful because it kept verification in the loop while the internal structure changed. The bridge kept the surrounding Python pipeline stable, the diff script exposed regressions quickly, and the Haskell refiner could then be reorganised into an explicit-state design without losing confidence in the generated output.



Chapter 5

Evaluation

This chapter evaluates the finished port against the main aim of the dissertation. It first explains how output comparison was carried out across the seven active repository models, then reports the differences that were found, including the one substantive task manager mismatch and its fix, and finally discusses what the experiment says about behavioural preservation and workflow compatibility.

5.1 Experiments

The evaluation was based on differential comparison between the original Python refiner and the Haskell refiner. For each of the seven active models in the repository, test code was generated once with the Python version and once with the Haskell version. The resulting files were then compared with a diff-based workflow, and each difference was classified as either semantic, where the generated C code changed meaning, or non-semantic, where only formatting changed.

5.2 Results

The comparison covered all seven active repository models: `barrier-mgr`, `chains`, `event-mgr`, `msg-mgr`, `sem-mgr`, `task-mgr`, and `proto-sem`. The number of traces compared for each model and the remaining difference type after the final fix are shown in Table 5.1.

Model	Trails	Remaining Difference After Final Fix
barrier-mgr	20	one extra blank line before the final SPDX footer
chains	21	one extra blank line before the final SPDX footer
event-mgr	11	one extra blank line before the final SPDX footer
msg-mgr	18	one extra blank line before the final SPDX footer
sem-mgr	36	one extra blank line before <code>// Task 0</code>
task-mgr	19	one extra blank line before the final SPDX footer
proto-sem	5	one extra blank line before the final SPDX footer

Table 5.1: Comparison results across the seven models after fixing the substantive `task-mgr` formatting bug.

This result is slightly more nuanced than simple text equality. Every compared file still differs textually, but after the final fix the remaining differences are all limited to harmless blank lines. For most models the extra line appears immediately before the final SPDX footer. For `sem-mgr` it appears just before the `// Task 0` section. These are therefore formatting mismatches rather than behavioural mismatches.

The one substantive difference found during evaluation appeared in `task-mgr` models 14 and 16. In those files the Python output contained `do { ... } while (...)` while the Haskell output contained `do {{ ... }} while (...)`, which is not valid C. The source of the bug turned out not to be the task manager model itself, but a mismatch between Python `str.format` semantics and the Haskell template formatter. The refinement dictionary stored literal C braces as doubled braces, and the Haskell formatter replaced placeholders without collapsing `{{` and `}}` back to single braces.

```
formatTemplate template args =
  let indexed =
    L.foldl'
      (\acc (i, arg) -> T.replace ("{" <>
tshow i <> "}") arg acc)
    template
    (zip [0 :: Int ..] args)
```

```
formatted = L.foldl' (\acc arg -> replaceFirst "{}" arg
acc) indexed args
in T.replace "}}" "}" (T.replace "{{" "{" formatted)
```

Listing 5.1: Brace escaping added to the Haskell template formatter

After rebuilding the Haskell refiner and replaying the saved `task-mgr .spn` files, the doubled-brace mismatch disappeared. Models 14 and 16 then reduced to the same blank-line-only difference seen elsewhere. This is an important result for the dissertation because it shows that the diff workflow did not merely report mismatches; it also isolated one concrete defect and guided it to a specific fix. 

5.3 Summary

The evaluation shows that the Haskell refiner preserves the behaviour of the Python version. One substantive task manager bug was found, traced to template brace escaping, and fixed. After that repair the remaining differences across all seven models were non-semantic blank lines.

Chapter 6

Conclusions & Future Work

This final chapter draws together the outcome of the dissertation and outlines the most useful directions for later work. It first restates what the project achieved, then reflects on the main limitation that remained after evaluation, and finally identifies the next technical steps that would make the port more complete.

6.1 Conclusions

This dissertation set out to examine whether the most difficult part of the testbuilder pipeline could be ported from Python to Haskell without losing the behaviour of the original system. The work showed that a gradual porting strategy was the correct choice for this problem. Because `refine_command` is stateful and tightly coupled to the rest of the pipeline, replacing it all at once would have carried a much higher risk of large regressions. The bridge and diff-based workflow reduced that risk by allowing the Haskell code to be checked continuously against the Python version.

The main contribution of the project is therefore not just a second implementation of `refine_command`, but a practical method for replacing part of an existing formal methods toolchain in a controlled way. The project also showed that the port only became worthwhile once it stopped imitating the Python structure line by line and started using more natural Haskell techniques, particularly explicit state handling and clearer decomposition of the refinement logic.

The evaluation strengthened the behavioural claim further by exposing one substantive `task_mgr` formatting bug, tracing it to template brace escaping, and fixing it. After that repair the remaining differences across the seven models were limited to whitespace. This means that the `has strpmg output preservation`, but not byte-for-byte textual parity.

6.2 Future Work

While not as crucial as the porting of the refine command itself, it was originally planned to port `spin2test` to Haskell as well. This would simplify the communication between `spin2test` and `refine_command` by removing the current dependence on standard input and output messages passed through the bridge. At present the bridge file and the refiner exchange commands and results continuously. In contrast, `testbuilder` launches `spin2test` once and then lets it read the necessary `.spn`, YAML, and template files itself. Moving this part of the system as well would reduce the amount of fragile cross-language coordination and likely make the whole pipeline easier to maintain.

Another useful piece of future work would be a more direct evaluation of maintainability. The clearest version of this would be to add a new annotation type or a new output target in both systems and compare the amount and complexity of the changes required. That was beyond the scope of this dissertation, but it would test the maintainability claim more directly.

In summary, the dissertation achieved a behaviour-preserving port of the refiner and established a practical method for migrating a stateful part of an existing toolchain without losing confidence in the output. The next step is to reduce the remaining workflow friction and test the maintainability more directly.



Bibliography

- Fisher, D. (2014). Hapy. GitHub repository. Haskell bindings for Python.
- Hambüchen, N. (2011). call-haskell-from-anything. GitHub repository. Call Haskell functions from other languages via serialization and dynamic libraries.
- Hudak, P., Hughes, J., Jones, S. L. P., and Wadler, P. (2007). A history of haskell: Being lazy with class. In *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages*. HOPL III.
- Hughes, J. (1989). Why functional programming matters. *The Computer Journal*, 32(2):98–107.
- Jaškuć, J. (2022). Spin/promela-based test generation framework for rtems barrier manager. Master’s thesis, School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland.
- Yang, H., Nong, Y., Wang, S., and Cai, H. (2024). Multi-language software development: Issues, challenges, and solutions. *IEEE Transactions on Software Engineering*, 50(3):512–533.
- Zheng, Y., Zhou, J., Krause, P., and Latifi, S. (2007). A model checking based test case generation framework for web services. In *Proceedings of the Fourth International Conference on Information Technology*, pages 715–720. Information Technology: New Generations.

